PART 6

Scaling & Edge

45 Autoscaling

46 Horizontal Scaling

47 Vertical Scaling

48 CDN

49 Edge Caching

50 Cache Invalidation

#045

Autoscaling

WHAT IT IS

Automatically adding or removing instances based on load.

WHY IT MATTERS

Handles spikes without paying for peak capacity all day.

EXAMPLE

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: taskflow-web }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind:
    Deployment, name: taskflow-web }
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type:
    Utilization, averageUtilization: 65 } }
```

#046

Horizontal Scaling

WHAT IT IS

Adding more machines/instances ("scale out").

WHY IT MATTERS

Practically unlimited growth and better fault tolerance.

EXAMPLE

1 server → 5 identical app servers behind a load balancer. Requirements: stateless app, shared Redis for sessions/cache, shared object storage (S3) instead of local disk, centralized logs.

#047

Vertical Scaling

WHAT IT IS

Giving one machine more CPU/RAM ("scale up").

WHY IT MATTERS

Simplest first step, no code changes; has a ceiling and is a single point of failure.

EXAMPLE

Upgrade the Postgres host from 4 vCPU/16GB to 16 vCPU/64GB and raise shared_buffers/work_mem. Often the right answer for databases before sharding.

#048

CDN

WHAT IT IS

A global network of edge servers that serve cached content close to users.

WHY IT MATTERS

Lower latency, less origin load, built-in DDoS absorption.

EXAMPLE

```
// config/filesystems.php: store uploads on S3,
    front with CDN
'url' => env('AWS_URL'), // https://cdn.example.com
```

#049

Edge Caching

WHAT IT IS

Caching full responses (or computing) at CDN edge nodes, not just static files.

WHY IT MATTERS

Public API/HTML responses return in milliseconds from the nearest region.

EXAMPLE

```
return response($html)->header('Cache-Control',
    'public, s-maxage=300, stale-while-revalidate=60'
    );
```

#050

Cache Invalidation

WHAT IT IS

Removing or refreshing cached data when the source changes.

WHY IT MATTERS

Famously hard; stale data is a bug, no caching is slow.

EXAMPLE

```
// Tags: flush a group (Redis/Memcached)
Cache::tags(["project:{$id}"])->remember('stats',
    600, fn () => $this->stats($id));
Cache::tags(["project:{$id}"])->flush();

// Event-based: bust on change
protected static function booted() {
    static::saved(fn ($t) => Cache::forget(
    "project:{$t->project_id}:stats"));
}
```

Found this useful?

Save it for later.

Share it with a teammate.

Full guide with all code examples:
the Backend Engineering Handbook (GitHub).